42Factors: a cloud-hosted shoe shop with sales and inventory analytics

Module INF2006 Cloud Computing & Big Data · Team project report · **Group** 21 · **Date** 11 October 2026

Members LIM CHEE LONG (2500584); MOHAMMED SYAHFARIZ BIN EFFENDY (2500717); MUHAMMAD ERFAN BIN MOHAN (2500722); JUN KEAT LIM (2501004); PHUA KIA WEN HERMANN (2501300)

Repository commit recorded in project_manifest.yaml at submission · Cloud AWS, us-east-1 (AWS Academy Learner Lab) · Evidence index project_manifest.yaml, evidence/README.md

How to read this report. The manifest (`project_manifest.yaml`) lists components, run commands and evidence paths; this report explains why those components were chosen, how they were tested and what the results mean. Each claim is followed by the evidence path that supports it. Raw command output lives under `evidence/raw/`, named `<CASE>_<date>`. AWS evidence was collected in two dated sessions: variant A (EC2 + Auto Scaling group) on 8 October and variant B (ECS on Fargate) on 9 October 2026 (UTC). Both ran the same pinned image digest and RDS class, and both compute stacks were torn down afterwards. Unless stated otherwise, comparative load, scale-out and replacement results are single runs (n = 1) and indicative.

1. Problem, users and success criteria

The problem. 42Factors is a small online sneaker shop in Singapore run by one store operator. It has two kinds of user with different needs. **Customers** need to buy a pair in their size and trust the outcome. When a popular model is restocked, many buyers try to take the same few units at once. A shop that oversells, charges twice on a retried click, or loses the cart when a server fails loses the sale and the customer's trust. The **store operator** needs to know what is selling and which sizes will run out before the next delivery can arrive. A small shop has no analyst, so the numbers have to be understandable without expertise and reproducible enough to trust.

The service is therefore an ecommerce workflow (register, browse 20 shoe models in 10 US sizes each, cart, simulated checkout, order history) plus an operator-only dashboard. The dashboard shows sales trends, top sellers, size demand, stock coverage in days and replenishment flags, and a descriptive customer segmentation. Payments are simulated and all data is synthetic. Every architecture choice in Sections 2 and 3 traces back to one of these two needs.

Assumptions and constraints. The expected peak is about 15 concurrent visitors, roughly 0.4 dynamic page views per second, with browsing far outnumbering purchases. This is an assumption based on public SME benchmarks, not measured traffic (`docs/plans/load-harness.md`). The shop serves one country, so it runs in a single region. The team was constrained by a four-week build, the Learner Lab (no custom IAM roles, a fixed region, credits capped at USD 50 per member) and a backend feature freeze from 1 October so that every test ran on one pinned container image.

#	Success criterion (fixed before testing)	Measure	Result	Evidence
SC1	Checkout stays correct: no overselling, no duplicate order on retry, no access to another customer's orders	Concurrency, idempotency and access tests; orders in load runs	Met: local correctness tests pass; all recorded A/B load runs had 0 stock conflicts and 0 errors	`SEC-01, SEC-05; LOAD runs`
SC2	Responsive at the declared load: dynamic p95 < 800 ms, checkout p95 < 2 s, unexpected errors < 1%	Locust saturation runs on one EC2 instance and one Fargate task	Partial: final B met the dynamic p95 limit at 8 no-think-time users (202 ms), with 0 errors through 512; at 16 users p95 was 1.204 s. Checkout p95 at declared load was not established	`LOAD-03`

#	Success criterion (fixed before testing)	Measure	Result	Evidence
SC3	Losing one web server loses no carts or logins; capacity grows without a person	Scale-out and graceful replacement runs; RES-04 request-log analysis	Partial (graceful removal, n = 1): A/B scale-out was automatic; two healthy units returned in 114 s/50 s with 0 errors; no session or cart failures among logged-in visits spanning the removal (A: 11, B: 3), but the logs do not record the serving unit	`RES-03a, RES-03b, RES-04`
SC4	Analytics are exactly reproducible	Independent pandas recomputation of the SQL refresh	Met: 0 mismatches over 41,345 daily rows	`DATA-01`
SC5	Data stores private, traffic encrypted, no secrets in code or logs	Cloud security cases and scans	Partial: storage, TLS and secret controls passed on A/B; WAF blocking passed on B. No MFA or app throttling; LabRole remains broad	`SEC-09-11, SEC-17-18; SCAN-01`
SC6	Proportionate cost; nothing left running unintentionally	Provisional cost model; teardown records	Partial: A/B compute stacks were destroyed and base resources retained intentionally; list-price inputs were checked on 9 October 2026, but excluded usage categories keep the estimate incomplete	`OPS teardown; cost model`

2. Solution overview and architecture

42Factors is a modular Django monolith with packages for accounts, catalogue, cart, orders and reporting. It renders server-side HTML with HTMX partial updates and runs under Gunicorn in one Docker image. The final AWS architecture is variant B: an Amazon ECS service on Fargate behind an Application Load Balancer, with Amazon RDS for PostgreSQL as the authoritative store for sessions, carts, orders and analytics summaries. Variant A (EC2 with an Auto Scaling group) was built and measured as the evaluated alternative. Both variants recorded zero errors; B was chosen for faster replacement (50 s vs 114 s), faster alarm-to-healthy scale-out (101 s vs 161 s) and no host patching, despite A's lower unit price and slightly higher throughput (indicative, n = 1; ADR-018). Figure 1 labels the AWS services; evidence/architecture.md maps each one to its Terraform resource (src/infrastructure/) and its test evidence.

42Factors on AWS, variant B: ECS on Fargate, us-east-1

Legend

Black solid: request path, numbered 1 to 4
Black dashed: scaling and admin
Purple solid: platform calls
Grey dotted: attached to the load balancer

Known limits

- RDS is single-AZ, no standby
- Tasks run in public subnets with public IPs, no NAT; AWS services are reached through the internet gateway
- Task and execution roles are the lab's LabRole
- WAF login rate rule counts by default; block mode tested on this variant
- Security groups: internet to sg-alb (443, 80); sg-app takes 8000 from sg-alb only; sg-db takes 5432 from sg-app only

Variant A (EC2 + Auto Scaling group) was measured as the alternative and is not drawn (ADR-018).

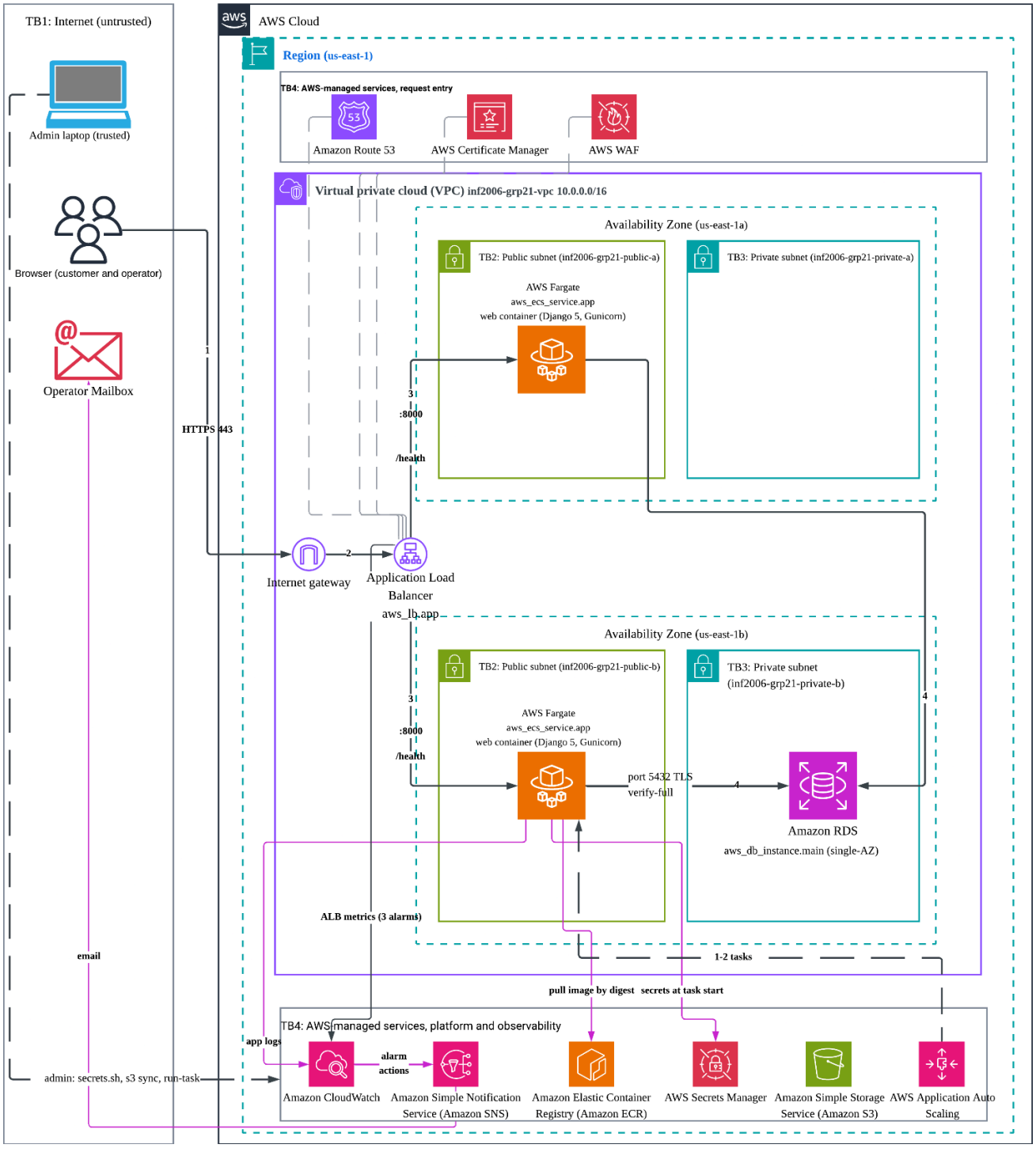


Figure 1. Final architecture, variant B (ECS on Fargate). The main request path is browser → Route 53/ALB → Fargate task → RDS. Boxes TB1–TB4 mark four trust boundaries; the two task icons show scaled-out placement (minimum one task); variant A (EC2) is not drawn. Source and label-to-evidence map: evidence/architecture.md.

Request path. A request reaches the ALB over HTTPS [1]; port 80 redirects to 443 and an AWS

WAF rate rule on POST /login/ counts by default (blocking demonstrated on B). The ALB routes traffic to healthy Fargate tasks on port 8000 (if every registered target is unhealthy, it fails open and routes to all of them). Application Auto Scaling changes desired task count from one to two against ALB request count [2]. Sessions and carts are stored in PostgreSQL, so tasks are replaceable and load-balancer stickiness is disabled [7]; the RES-04 request-log analysis found no session or cart failures among logged-in visits spanning a graceful removal (A: 11, B: 3; small sample), though the logs do not record which unit served each request. Checkout remains one database transaction (Section 4). Static files and 3D models are packaged in the image and served by WhiteNoise, so browsers do not read S3.

Data path. The raw synthetic dataset is preserved and synced to a private S3 bucket. An authorised one-off ECS job can download and validate it before loading RDS (tested offline; the recorded AWS import used variant A); other one-off tasks run bootstrap, migrations, checks, operator creation and analytics refresh. The dashboard reads versioned summary tables rather than aggregating raw order lines during a request. Customer segmentation runs offline on the team laptop and is loaded as a removable batch; it was not run on AWS.

Operations path. Terraform separates the persistent base/ stack (VPC, private RDS, ECR, S3 controls, Secrets Manager, DNS, ACM and SNS) from the disposable fargate/ web stack (ALB, WAF, ECS service, scaling, logs and alarms). Fargate tasks sit in two public subnets because the lab build avoids a NAT gateway, but sg-app admits port 8000 only from sg-alb; RDS sits in two private subnets and sg-db admits 5432 only from sg-app. Images are pulled from ECR by digest, tasks read secrets at startup, and CloudWatch/SNS carry logs, alarms and email notifications. The web stack was torn down after the 9 October session while base data resources were retained deliberately. Evidence: evidence/architecture.md; ADR-018.

3. Cloud service/deployment choices and trade-offs

Service and deployment model. The final compute platform is Amazon ECS on Fargate: the team owns one immutable Django container image while AWS manages the hosts, so the service is PaaS-leaning rather than VM-level IaaS [3]. It runs in the public AWS cloud, in us-east-1 because of the Learner Lab, across two Availability Zones. The ALB and Fargate tasks are replaceable; RDS PostgreSQL is the authoritative state store and S3 holds the accepted dataset. A production Singapore deployment would retain the topology but use ap-southeast-1. Evidence: ADR-018 and evidence/architecture.md.

Variant comparison. Both variants used the same image digest, db.t4g.small database, two vCPUs/four GiB of compute and load harness. Each condition was run once, so differences are indicative (n = 1).

Measure	**A: EC2 + ASG**	**B: ECS/Fargate**	**Reading**
Errors across recorded runs	0	0	Tie
Graceful replacement to healthy	114 s	**50 s**	B recovered faster
Scale-out, alarm to healthy	161 s	**101 s**	B scaled faster
One-unit capacity	**46.5 actions/s; 118.8 requests/s**	44.0 actions/s; 105.5 requests/s	A had the higher ceiling
Compute list price per unit-hour	**USD** 0.085	USD 0.099	A was cheaper
Operational burden (team judgement)	7/10; patch hosts, AMI and SSM path	**9/10; no hosts to patch**	B suited the small team

Decision. The weights were fixed before testing: operational burden 25%, replacement/error window 25%, latency 20%, cost 15%, scale-out 10% and capacity 5%. ADR-018 applies these criteria and selects B: for a small Singapore shop with a one-to-three-person team, AWS managing the hosts outweighs EC2's lower price and greater host control. The planned latency measure (p95 at 10 and 100 users) was not run, so latency was scored from a low-load comparison (a tie); dropping it and renormalising still gives B 9.6 vs A 8.0. With one repeat per condition, the comparison is indicative rather than statistical proof: removing the

operational-burden and replacement advantages makes the result effectively a tie, while cost alone favours A. Full calculation and sensitivity: ADR-018; measurements: evidence/test-performance.md and evidence/load-summary.md.

Alternatives considered. AWS Lambda with API Gateway was rejected because Django is a long-running process that reuses persistent database connections (CONN_MAX_AGE = 60): many concurrent functions would each open a PostgreSQL connection exactly when stock rows are locked at a restock, and adapting and re-validating the frozen application for Lambda did not fit the four-week build. Amazon EKS adds a control plane for a single deployable, well beyond what a one-to-three-engineer team needs. Elastic Beanstalk and App Runner hide the network and scaling decisions this project must show, and the Learner Lab restricts some of their roles. ECS on EC2 was dropped for time. EC2 with an Auto Scaling group (variant A) was built and measured as the closest alternative (ADR-018).

Deployment trade-offs.

- **Public application subnets, no NAT gateway:** Fargate tasks receive public IPs for outbound AWS API access, but sg-app accepts port 8000 only from sg-alb and there is no SSH. This avoids NAT cost but is weaker defence in depth.
- **Private Single-AZ RDS:** sg-db accepts 5432 only from sg-app and TLS is forced. One saturated task used at most 9.09% RDS CPU, but the database remains a single point of failure. We did not run a second database: a read replica spreads reads and gives no automatic failover, the frozen application has no read routing, and RDS CPU never exceeded 10.46% with at most two connections. Multi-AZ is the setting that fits availability (one setting, roughly twice the instance charge); it was not enabled for cost and time and no failover was tested, so no availability claim is made. Automated backups are configured for 7 days; no restore was tested.
- **No ElastiCache:** measurements locate the limit in the application tier, not RDS, so a cache would add cost and staleness without demonstrated benefit.
- **Static assets in the image:** WhiteNoise keeps deployment simple, but the load harness did not fetch the roughly 15 MB of 3D models; S3 and CloudFront are a future scale option, not part of this build.
- **Request-count target tracking:** Application Auto Scaling uses 300 ALB requests per target per minute and changes desired task count from one to two. B took 407 s from the load step to a second healthy task, so a known campaign should be pre-scaled.

Lab constraints. The lab fixes us-east-1 and denies custom IAM roles, so every AWS workload uses the broad LabRole; least-privilege IAM remains an acknowledged gap. The S3 bucket was created by CLI because a denied read prevents Terraform from creating it, while its six security settings remain Terraform-managed. Evidence: evidence/aws-build-runbook.md.

4. Implementation, data design and security controls

Checkout correctness

Checkout protects the two failure modes most damaging to the shop: overselling and duplicate orders. The review page renders a SHA-256 fingerprint of the displayed cart lines; the POST carries that fingerprint and a user-scoped idempotency key. In one database transaction the service locks product rows in a consistent order, re-checks stock, derives prices and the owner on the server, creates the order and lines, and decrements stock. A database constraint forbids negative stock, while the unique (user, idempotency key) constraint makes an equivalent retry return the original order; reusing the key for a changed cart returns 409. Carts never reserve stock, and every cart or order lookup is owner-filtered. SEC-01 and SEC-05 record the access-control, replay and concurrent-checkout results; contracts: docs/API.md and docs/SCHEMA.md.

Data design

The schema separates imported history, live commerce and reporting state. Imported orders use a namespace/external ID and are immutable; live orders use the idempotency key. Product stock is current operational state, while sealed InventorySnapshot rows preserve stock at a reporting cutoff. Each AnalyticsRun owns its cutoff, inventory version and DailyProductSales rows, so the dashboard exposes one consistent completed run. The refresh uses a PostgreSQL advisory lock to prevent two runs interleaving.

Evidence and contracts: docs/SCHEMA.md, docs/DATA_PIPELINE.md and evidence/test-data-ai.md (DATA-01).

Release and configuration

Each release is one immutable image in ECR, and the final ECS task definition pins it by digest; all A/B evidence used sha256:fa72e949b47d…. Fargate web tasks read only the web secret at startup through AWS_SECRET_ID and stop on a missing or malformed payload—no secret value is stored in the task definition. Bootstrap, migration, import, operator creation and refresh use one-off ECS run-task commands with separate secret selections; on B, check ran in AWS, while import/operator/all were verified offline, so the report does not claim otherwise. Database roles separate the migration owner, restricted runtime login and read-only trainer. A fresh-clone rehearsal of docs/SETUP.md rebuilt the system and recorded 497 passed, 8 skipped; the Python 3.14 analytics-wheel failure was resolved by the documented Python 3.13 container path. Evidence: evidence/raw/REPRO-01_2026-10-09_linux_clean-clone.txt and evidence/architecture.md.

Security controls

Nineteen threats from OWASP Top 10:2025 [8], OWASP Automated Threats [9] and the Verizon 2025 DBIR retail snapshot [10] are mapped to controls and evidence in `evidence/threat-control-map.md`. The table summarises the controls tested on AWS and the residual gaps.

Threat	Control	Test and result
Data store exposure (T9)	RDS in private subnets, not publicly accessible, encrypted; S3 Block Public Access; SG chain ALB→app:8000→DB:5432	SEC-09 PASS on A/B: RDS is private and encrypted; the database admits 5432 only from the app group; S3 blocks public access and anonymous requests return 403
Interception (T10)	ACM certificate, HTTP→HTTPS 301, HSTS, Secure cookies; verify-full TLS to RDS	SEC-10 PASS on A/B: HTTP redirects to HTTPS, the listener enforces TLS 1.2+, the certificate verifies, and RDS uses verify-full
Secret exposure (T8)	Secrets Manager at task start; secret names only in task configuration; no secret values in the image, Git or logs	SEC-11 and SEC-18 PASS on A/B: tasks retrieved the runtime secret and recorded logs contained 0 secret-pattern matches. SCAN-01 history scan passed; reviewed raw-evidence matches remain documented
Access control, CSRF, tampering, XSS (T1–T6)	Owner-filtered queries, operator permissions, CSRF on every mutation including HTMX, server-side prices, auto-escaping	SEC-01 to SEC-06 PASS, part of the 25-test local security suite SEC-01 to SEC-08 (rerun 1 Oct)
Credential stuffing (T7)	Password validators; WAF rate rule on `POST /login/` (count mode)	Partial: SEC-17 passed on B (24 login attempts allowed, then 5 blocked; browsing unaffected). Default deployment mode is count; no MFA or application throttling
Over-privileged identity (T19); script injection (T13); base image (T18)	Network and database controls; vendored JS with recorded hashes; pinned digests	Accepted / partial: LabRole is broad; no Content-Security-Policy; the local image scan found 279 OS findings (5 critical, 61 high; 258 with no Debian fix) and 6 in pip's vendored packages; 21 need a newer base digest, held back by the freeze. Reachability was not analysed and the pushed ECR digest was not scanned (evidence/security-scan-triage.md)

5. Analytics or AI/ML feature: data, method, evaluation and limitations

Data. The source is the Analytics Engineering *Ecommerce Sales & Returns* dataset v1.0.0, fully synthetic and licensed CC BY 4.0, downloaded on 22 September 2026 [6]: 20,000 customers, 100,000 orders and 237,537 order lines. The raw files are committed unchanged with SHA-256 checksums (`data/raw/checksums.txt`). Integrity profiling found all six published row counts exact and zero rejected rows (`data/PROFILE.md`). The source sells general goods and has no stock, so two reproducible, seeded scripts adapt it. `analytics/build_shoe_dataset.py` re-points the same customers, orders and lines to 200 shoe products (20 models × 10 sizes) and recomputes line prices. The importer generates stock and supplier lead times and seals them as an inventory snapshot at the reporting cutoff. Both are documented in `data/README.md`.

Method: deterministic analytics. For an as-of date D, `refresh_analytics` aggregates completed order lines at their historical prices by Singapore business date and product, over the 366 days ending at $L = D - 1$. From these rows the dashboard derives daily and category sales, top models, size demand, and three planning figures. **Average daily sales** is units over the last seven complete days divided by seven, zero-sale days included. **Stock coverage** is snapshot stock divided by average daily sales. A size is **flagged** when coverage is at or below its lead time. For example, 35 units in seven days is 5 a day; 30 in stock is 6 days of cover; a 7-day lead time means the size is flagged for a stock check. Zero sales give "coverage unavailable", never infinity, and fewer than seven days of history gives "insufficient history" rather than a smaller denominator (`docs/ANALYTICS.md`).

Method: descriptive ML. analytics/segment_customers.py fits K-means [13] to standardised log1p recency, log1p frequency and log1p average order value (RFM). It uses completed orders before the cutoff, excludes privileged accounts, evaluates k = 2 to 4 by silhouette and fits the full eligible cohort with a fixed seed. Training runs offline, outside the web image, through the read-only trainer connection. A separate strict import binds the JSON export to one completed AnalyticsRun and recipe version, so removing the ML leaves checkout and the deterministic analytics untouched.

Evaluation. The analytics were checked by an independent implementation rather than by re-running the same SQL. `analytics/verify_refresh.py` recomputes every daily row, unit, amount, coverage value and flag from the raw tables in pandas and compares them exactly.

Check	Result	Evidence
SQL refresh vs independent pandas recomputation (shoe dataset)	41,345 = 41,345 daily rows; 0 row, unit, amount and flag mismatches; max monetary difference 0.00	`evidence/test-data-ai.md DATA-01`
Same refresh job on Variant A AWS RDS, 8 Oct	41,345 daily rows from 71,079 completed lines in 3.1 s; execution counts matched local, but the independent pandas recomputation was not run against RDS	`evidence/raw/DATA-01_2026-10-08_aws_refresh.txt`
Segmentation stability (k = 2 selected)	Silhouette 0.2743; seed ARI 0.9976; 20 × 80% subsamples gave ARI mean 0.976 and min 0.9545; smallest segment 42.6% (criteria ≥ 0.90 and ≥ 5%, set beforehand)	`evidence/ml-holdout-evaluation.json; evidence/raw/DATA-02_2026-09-26_m03-train-load.txt`
Does the segment predict later buying? (90-day time split)	No. With the adopted recipe the split selected k = 4; repurchase ranged 29.99%–30.98% (lift 0.978–1.010) versus 30.66% overall	`evidence/test-data-ai.md (M-02)`

The published full-cohort run has two descriptive groups: "recent, more orders, higher spend" (11,338 customers; 77.5% of sales) and "older, fewer orders, lower spend" (8,428; 22.5%). All 19,766 assignments loaded and the displayed shares sum to 100%. The separate 90-day evaluation refitted the adopted recipe on pre-split history and selected four clusters; none separated later purchasing. Past frequency correlated 0.00 with future orders because the generator draws them independently. The dashboard therefore labels the groups as descriptive; they must not drive churn or targeting decisions on this data.

Limitations. Synthetic data shows that the mechanism works, not what real demand looks like. Weekly patterns are weak, stock and lead times are generated, and 9 of 200 products had no recent sales, so their coverage is unavailable. Totals are completed-sales value, not accounting revenue: refunds, tax and shipping are excluded. Refresh remains operator-triggered: the recorded AWS run used Variant A through Systems Manager; the final Variant B one-off ECS refresh path was tested offline only, and no scheduler was built. ML training and loading were local only, not run on AWS. Lifetime RFM also favours long-tenure customers because tenure is not modelled (M-04).

6. Testing, scalability/resilience and monitoring results

Test strategy. Evidence is split into local correctness, deployed workflow, load/recovery and monitoring. On real PostgreSQL, FUNC-01's 93 core workflow tests and the 25 security tests of SEC-01 to SEC-08 passed; the latest recorded full suite was 497 passed with 8 opt-in bootstrap skips. The FUNC-02 browser walkthrough covered purchase, operator and access-denial flows. On AWS, identical FUNC-03 smoke visits passed on A and B: two orders and 0 errors on each. No cloud browser screenshots were retained. Every case records its objective, setup, pre-written expectation, actual result, date and raw artefact (evidence/README.md).

Load method. Locust [12] ran from a separate c5.large in us-east-1, using seed 42, 2,200 accounts and a 90% browsing / 7.5% cart / 2.5% buying mix. A and B used the same image digest, db.t4g.small RDS, two-vCPU/four-GiB application unit and WAF count mode. One valid scale-out, replacement and saturation run was recorded per variant; generator CPU's rolling peak was 20%, so it was not the limit. tests/load/analyze.py rebuilds evidence/load-summary.md from the raw folders. Comparisons are indicative because n = 1, the serving CPUs differed, and database cache and credits could not be reset between days.

Capacity comparison (one application unit; n = 1 each)

Measured comparison. With no think time, A met the p95 < 800 ms threshold through 16 users (783 ms; 46.54 page actions/s); B met it through 8 users (202 ms; 44.01 actions/s) and reached 1,204 ms at 16. Both recorded 0 request errors through 512 users. Maximum throughput was 118.8 requests/s on A and 105.5 on B (evidence/test-performance.md LOAD-03; evidence/load-summary.md).
On both variants throughput flattened while latency climbed: the application unit, not RDS, was the bottleneck. Application CPU reached about 94–96%, whereas RDS CPU stayed at or below 10.46% on A and 9.09% on B, connections never exceeded two and memory stayed stable. Both practical capacities are about 100 times the assumed 0.4-page-action/s peak, so capacity did not decide the architecture. At 512 users p95 reached 13.9 s on A and 14.4 s on B; health checks failed briefly despite 0 request errors. The result shows queued overload, not acceptable customer latency, and it does not justify a cache or larger database.

Scaling and recovery (one run per variant; n = 1 each)

Run	Measure	A: EC2 + ASG	B: ECS/Fargate
RES-03a scale-out: 10 → 200 users with 36 s think time, 1–2 units	Load step to target-tracking alarm	299 s	305 s
	Alarm to scaling action (desired 1 → 2)	64 s	67 s
	Alarm to new unit healthy in the target group	161 s	101 s
	Load step to second healthy unit · errors · ALB p95 max	460 s · 0 · 0.67 s	407 s · 0 · 0.27 s
RES-03b replacement: 2 units, 100 users, one removed gracefully (A: ASG instance termination, 8 Oct 17:29:38Z; B: ECS task stop, 9 Oct 03:17:34Z)	Removal to detection	2 s	18 s
	Removal to two healthy units again	114 s	50 s
	Error window · failed requests · ALB p95 max	0 s · 0 of 7,474 · 0.63 s	0 s · 0 of 7,474 · 0.05 s
RES-04 session continuity	Logged-in visits spanning	11 · 29 of 29 · 0	3 · 8 of 8 · 0

Run	Measure	A: EC2 + ASG	B: ECS/Fargate
(from the RES-03b request log)	the removal · later requests ok · second logins		

Both variants scaled from one to two units without a person or an error. The target was a conservative 300 requests per target per minute (5 requests/s), so scale-out fired at about 14% CPU (14.56% on A, 14.04% on B); it did not demonstrate scaling out of CPU saturation. From the load step to two healthy units, A took 460 s and B 407 s; alarm-to-healthy was 161 s versus 101 s. Under 100 users, a gracefully removed unit was replaced in 114 s on A and 50 s on B, with 0 failed requests out of 7,474 on each. These tests show load-balancer draining and planned replacement, not a sudden crash. Scale-in was observed only on B after the run. No session or cart failures were observed among logged-in visits spanning the graceful removal (A: 11, B: 3; RES-04); the logs do not record the serving unit, so continuity across a confirmed unit transition was not established. B's faster recovery and lack of host maintenance supported its selection, while A retained the slightly higher ceiling and lower price (ADR-018).

Monitoring

The application emits structured request logs to CloudWatch without passwords, cookies or query strings; MON-01's status-by-time query completed on A and B. In MON-02, the recent-errors query completed, but the second saved query was rejected as malformed on both variants; its Terraform text was corrected later but not applied or re-run. MON-03 only partially records the email path: A retains the ALARM email leg and B the OK leg, so no single variant has both. MON-04 is stronger operational evidence: real rising load triggered target-tracking alarms and scale-out actions on both variants. Remaining gaps are no RDS alarms or tracing and email-only notification.

Not tested, stated plainly. There were no repeat runs, so A/B differences are indicative (n = 1). The 10/100/1,000-user ladder, 1,000-user RDS pilot, Singapore latency run, Multi-AZ failover, sudden crash, interrupted-checkout recovery, and the local RES-01/02 database-outage cases were not completed. The harness also excluded the 3D models. SEC-17 block mode ran on B only and is reported in Section 4. A cloud build by a second person in another account was not performed; it needs a domain that person owns.

7. Cost, sustainability and operational considerations

Running cost. The selected minimum stack is one Fargate task (2 vCPU/4 GB), one ALB, one private Single-AZ db.t4g.small RDS and three public IPv4 addresses. tests/load/prices.json records verified us-east-1 on-demand prices checked on 9 October 2026 [4]. The four priced components total USD 0.16824/hour or USD 122.82/month (730 h), excluding storage, data transfer, ALB LCUs, WAF (about USD 6/month for one web ACL and one rule, plus requests), CloudWatch, Secrets Manager and Route 53.

Item	USD/hour	USD/month (730 h)
Fargate task: 2 vCPU / 4 GB × 1 (second task only while scaled out)	0.09874	72.08
Application Load Balancer (excluding capacity units)	0.0225	16.43
RDS `db.t4g.small` Single-AZ (excluding storage)	0.032	23.36
Public IPv4 addresses × 3 (task, two ALB nodes)	0.015	10.95
Storage, transfer, ALB LCUs, WAF (about USD 6/month plus requests), CloudWatch, secrets and Route 53	—	not included
Tracked components only	**0.16824**	**122.82**

Measured and variable cost. At the Fargate ceiling, task compute alone is about USD 0.00026 per 1,000 requests; at normal low traffic, fixed RDS, ALB and address charges dominate. The Learner Lab showed USD 9.80 used out of USD 50 at the end of the recorded cloud work. Usage increased by approximately USD 2.60 across the two evidence sessions, including both variants and the shared base. The web stacks and generator

were destroyed after sessions while base remained; teardown records verify no load balancer, instance, ECS cluster or NAT gateway remained (evidence/raw/OPS_2026-10-08_ec2_down.txt; evidence/raw/OPS_2026-10-09_fargate_down.txt).

Sustainability. One 2-vCPU/4-GB Fargate task delivered 44.01 page actions/s at practical capacity, about 100 times the assumed peak, so continuous full-size operation would waste capacity. The next step is to reduce task CPU/memory and repeat LOAD-03; a second task is for availability, not current demand. Seven-day log retention, precomputed analytics, batch-only ML and tearing down Fargate/loadgen reduce idle work [5]. The lab forced us-east-1; production would use ap-southeast-1 for customer latency and locality. Energy and carbon were not measured, so these are operational improvements, not a quantified sustainability result.

Operations. The final workflow is up.sh fargate, smoke.sh fargate and down.sh fargate. Migrations, import and analytics refresh run through jobs.sh fargate as one-off ECS tasks; there is no SSH or host operating system to patch. Operators diagnose failures through CloudWatch Logs and ECS service events. Releases use immutable image digests, so rollback redeploys the prior digest; a backend change creates a new digest and supersedes old evidence. Risks remain: Single-AZ RDS, broad lab-supplied LabRole, Terraform state on one apply device, a fresh-account Fargate bootstrap tested offline but not rerun from nothing on AWS, and a cloud-reproduction prerequisite: another account needs its own domain or a delegated subdomain (about USD 1 to 3 for the first year) because the application enforces HTTPS. No second-account build is recorded. The runbook and exact commands are in evidence/aws-build-runbook.md and src/infrastructure/README.md.

8. Team contribution, ethical considerations and reflection

Contribution. The team has five members. Two members led the cloud, backend and data work; three led the storefront, application testing, evidence and report. Responsibilities are recorded by role, artefact and evidence ownership below. The final contribution record, including per-member artefacts, evidence ownership and reflections, is in TEAM_CONTRIBUTIONS.md.

Member	Role	Main artefacts
Muhammad Erfan Bin Mohan (2500722)	Cloud lead, backend, load testing	Terraform for both compute variants (`src/infrastructure/`), load harness (`tests/load/`), authentication, order history and checkout fixes, ADRs, manifest. Owns the AWS deployment, load, scaling, resilience, monitoring and AWS security evidence.
Lim Chee Long (2500584)	Assistant cloud, backend	Local Docker/PostgreSQL foundation, dataset import with sealed inventory, operator dashboard, ML segment import and RFM evaluation, secrets loading, database TLS, role separation. Owns the review records in `docs/reviews/`.
Mohammed Syahfariz Bin Effendy (2500717)	Application testing, UI/UX, report	ML hold-out evaluation and fixes (`analytics/`), manage-products filters, product detail page, stay-on-page cart. Chose which ML review findings to act on and set the adoption criteria. Owns the data/AI evidence and application and UI testing.
Jun Keat Lim (2501004)	Report, UI/UX	Storefront redesign with dark mode and motion, 3D shoe models, 42Factors branding, shoe catalogue with sizes, analytics pagination. Owns frontend tests (`tests/test_frontend.py`) and storefront visual checks.
Phua Kia Wen Hermann (2501300)	Application testing, report	Review of the local functional and security test records (run by AI agents), FUNC-02 browser walkthrough of the storefront and dashboard, a reproduction test of the submission package that exposed the domain prerequisite for the cloud build, and the stay-on-page Add to cart request.

AI use. AI coding assistants were used extensively: Claude Code, OpenAI Codex agents, GitHub Copilot and Gemini. Tool use, licences and verification are recorded in AI_USE_DECLARATION.md. Third-party asset attributions, including the Dior model's CC BY 4.0 source, are recorded in src/static/models/CREDITS.md. The team chose the scope and made the design decisions. AI assistants also drafted and revised report text, which the two developers checked against the evidence. AI-assisted work was accepted through the applicable testing and evidence gates; completed independent reviews are under docs/reviews/, while the backend

review of later changes and the review of the ec2/ stack remain open; automated, deployed, database and browser checks are cited by the relevant evidence records. Quantitative claims cite saved raw output where applicable, and evidence identifies whether a check was agent-run or human-run.

Ethical considerations. *Data:* only synthetic data is used, from a dataset whose publisher states it is fully synthetic, under CC BY 4.0 with attribution preserved; no real personal data was collected, and imported customers get unusable passwords so they can never log in. *Profiling:* customer segmentation is a form of profiling, so the dashboard shows customer references rather than names or emails, the segments are labelled descriptive, and the null predictive result is published rather than hidden, so the feature is not used to target or exclude customers on evidence it does not have. A real deployment would also need a lawful purpose and notice under Singapore's PDPA [11]. *Security and honesty:* evidence is redacted of account IDs, IP addresses and email addresses; secrets never enter Git or Terraform state; and unmet or untested items are reported as such.

Individual reflections. Each member's final reflection follows and is also recorded in TEAM_CONTRIBUTIONS.md.

Muhammad Erfan Bin Mohan. I led the AWS side: the Terraform for both compute variants, the Locust load harness and the 8 October evidence session. This was my first exposure to Terraform and to working with real AWS services, and I am thankful for the chance to build, deploy and measure a real system rather than only read about one. The saturation run taught me to read the bottleneck from the metrics instead of guessing: instance CPU reached 94.6% while RDS stayed under 11%, so scaling the web tier was the right lever and a cache would have been wasted effort. I also learned to design around the Learner Lab limits, such as no custom IAM roles, rather than fight them. Hermann's reproduction test exposed a mistake of mine: the build script defaulted to our own domain, which another account cannot validate without control of that domain's DNS, so the cloud stack was reproducible only by me. In future I will treat anything owned outside the repository, such as a domain, an account or a name, as an explicit, documented input that is validated at the start and never defaulted, and I will have someone rebuild from a clean account early, not on the last day. Next time I would also deploy and measure the first variant end to end in week two, leaving time for repeat runs and for the remaining resilience gaps.

Lim Chee Long. I built the local Docker and PostgreSQL foundation, the dataset import with sealed inventory, the operator dashboard, and the secrets, database TLS and role-separation work. The most useful design lesson was keeping live stock and sealed snapshots apart, so a coverage figure never divides today's stock by last year's sales. Reviewing AI-written code against written contracts caught problems that passing tests alone would not have shown. Next time I would write the contract for each module before generating its code, and add a scheduled analytics refresh, which we left as a manual step.

Mohammed Syahfariz Bin Effendy. I worked on the ML hold-out evaluation, the manage-products, product detail and cart features, and the application and UI testing, and I chose which review findings to act on and set the adoption criteria. The main lesson was that the segments describe past customers faithfully but do not predict repurchase on this synthetic data, and that reporting this was better than tuning until a difference appeared. Deciding which AI review findings to fix, and which to keep as stated limitations, showed me that accepting every suggestion is not the same as good engineering. Next time I would run the hold-out test as soon as the first segments exist, so we learn early whether a feature earns its place.

Jun Keat Lim. I redesigned the storefront with dark mode and motion, added the 3D shoe models and the 42Factors branding, and built the shoe catalogue with sizes. I learned that every third-party asset needs its source and licence recorded at the moment it is added: the models are mostly CC BY with two non-commercial. Recovering the missing Dior model attribution before submission showed why the register must be complete. I also saw that front-end polish still needs tests, which is why the frontend test file exists. Next time I would keep an asset register from the first model I add and check page weight earlier.

Phua Kia Wen Hermann. I reviewed the local functional and security test records, ran the FUNC-02 browser walkthrough of the storefront and dashboard, and asked for the stay-on-page Add to cart behaviour. I learned to write the expected result before running a test and to keep the raw output, so that someone who did not run it can check it, and that even the Gemini-assisted dashboard description had to be cross-checked against the database before we recorded it as matching. I also ran a reproduction test of the submission package on my own machine, following the README; no raw log of that run was kept. The recorded local rehearsal (REPRO-01) is separate: an agent-run fresh clone on the team lead's laptop, which built, started,

imported the dataset, refreshed analytics and passed the application and analytics tests using the documented Python 3.13 container path. When I then tried to build the cloud stack in my own AWS account, it stalled: the team's domain points at Erfan's AWS account, so a certificate requested from another account is never approved unless its validation record is added to that domain's DNS. This was a prerequisite the team had not stated. It led to the domain requirement being documented, to a guard in the build script and to a subdomain-delegation helper, and it is why the cloud path is described as reproducible only by someone who controls a domain. If we repeated the project I would rerun the local test records against each new build earlier and automate more of the walkthrough. I also found an intermittent local “Internal Server Error”, which an AI-assisted analysis attributes to Gunicorn’s idle-connection timeout (an explanation, not reproduced with a saved log), and recorded it as a README limitation. I made the video for the submission.

References

[1]	Amazon Web Services. Create an HTTPS listener for your Application Load Balancer. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-https-listener.html (accessed 9 Oct 2026).
[2]	Amazon Web Services. Target tracking scaling policies for Application Auto Scaling. https://docs.aws.amazon.com/autoscaling/application/userguide/application-auto-scaling-target-tracking.html (accessed 9 Oct 2026).
[3]	Amazon Web Services. Architect your solution for Amazon ECS. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-configuration.html (accessed 9 Oct 2026).
[4]	Amazon Web Services. AWS Pricing Calculator. https://calculator.aws/ (prices checked 9 Oct 2026).
[5]	Amazon Web Services. AWS Well-Architected Framework: Sustainability Pillar. https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/ (accessed 9 Oct 2026).
[6]	Analytics Engineering. Ecommerce Sales & Returns Dataset (Version 1.0.0) [Data set]. https://www.analyticsengineering.com/datasets/ecommerce-sales (accessed 22 Sep 2026).
[7]	Django Software Foundation. How to use sessions. https://docs.djangoproject.com/en/5.2/topics/http/sessions/ (accessed 9 Oct 2026).
[8]	OWASP Foundation. OWASP Top 10:2025. https://owasp.org/Top10/2025/ (accessed 9 Oct 2026).
[9]	OWASP Foundation. Automated Threats to Web Applications. https://owasp.org/www-project-automated-threats-to-web-applications/ (accessed 9 Oct 2026).
[10]	Verizon. 2025 Data Breach Investigations Report: Retail Snapshot. https://www.verizon.com/business/resources/infographics/2025-dbir-retail-snapshot.pdf (accessed 9 Oct 2026).
[11]	Personal Data Protection Commission Singapore. Data Protection Obligations. https://www.pdpc.gov.sg/overview-of-pdpa/the-legislation/personal-data-protection-act/data-protection-obligations (accessed 9 Oct 2026).
[12]	Locust contributors. Locust 2.46 documentation. https://docs.locust.io/en/stable/ (accessed 9 Oct 2026).
[13]	Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Appendix A. Claim-to-evidence map

Claim (section)	Evidence path
Architecture decisions and alternatives (2, 3)	`docs/decisions/README.md`; `docs/decisions/ADR-018-final-architecture-ecs-fargate.md`; `evidence/architecture.md`
Infrastructure as code; build record (2, 3, 7)	`src/infrastructure/README.md`; `evidence/aws-build-runbook.md`
Functional tests (6)	`evidence/test-functional.md`; `evidence/raw/FUNC-01_2026-09-26.txt`; `evidence/raw/FUNC-03_2026-10-08_ec2_summary.txt`; `evidence/raw/FUNC-03_2026-10-09_fargate_summary.txt`
Security cases and scans (4)	`evidence/test-security.md`; `evidence/threat-control-map.md`; `evidence/security-scan-triage.md`
Data provenance and validation (5)	`data/README.md`; `data/PROFILE.md`; `data/raw/checksums.txt`; `evidence/test-data-ai.md`
ML evaluation (5)	`evidence/ml-quality-evaluation.json`; `evidence/ml-holdout-evaluation.json`; `evidence/raw/DATA-02_2026-09-26_m03-train-load.txt`
Load, scaling, replacement (6)	`evidence/test-performance.md`; `evidence/test-resilience.md`; `evidence/load-summary.md`; `tests/load/analyze.py`
Session continuity (6)	`evidence/raw/RES-04_2026-10-10_ec2_continuity.txt`; `evidence/raw/RES-04_2026-10-10_fargate_continuity.txt`; `tests/load/continuity.py`
Monitoring and alarm (6)	`evidence/monitoring.md`; `evidence/raw/MON-01_2026-10-08_ec2_sample.txt`; `evidence/raw/MON-01_2026-10-09_fargate_sample.txt`
Teardown and cost (7)	`evidence/raw/OPS_2026-10-08_ec2_down.txt`; `evidence/raw/OPS_2026-10-09_fargate_down.txt`; `tests/load/prices.json`
Local reproduction (all)	`README.md`; `docs/SETUP.md`; `project_manifest.yaml` (`run.commands`)
Contributions and AI use (8)	`TEAM_CONTRIBUTIONS.md`; `AI_USE_DECLARATION.md`; `docs/reviews/`